AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Michał Tokarz

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

2. Analiza problemu **4**

3. Projektowanie **6**

4. Implementacja **9**

4.1. Paradygmat obiektowy i architektura plików 9
4.2. Realizacja wzorca projektowego Fabryka 9
4.3. Nawigacja po strukturze – wzorzec Iterator 10
4.4. Zarządzanie powiązaniami – operacje wstawiania 10
4.5. Zarządzanie pamięcią operacyjną i usuwanie elementów 12
4.6. Zestawienie wydajności operacji 13
4.7. Powstałe wyniki i zastosowanie narzędzi pomocniczych 13

5. Wnioski **16**

Literatura **17**

Spis rysunków **17**

Spis tabel **18**

Spis listingów **19**

1. Ogólne określenie wymagań

Celem niniejszej pracy jest zaprojektowanie oraz implementacja dwukierunkowej listy powiązanej w języku C++, operującej na stercie. Głównym założeniem projektowym jest praktyczne wykorzystanie paradygmatu programowania obiektowego oraz wzorców projektowych, w szczególności wzorca Fabryki (Factory) do zarządzania tworzeniem obiektów oraz wzorca Iteratora do bezpiecznej nawigacji po elementach kolekcji. Równie ważnym celem edukacyjnym jest zapoznanie się z systemem kontroli wersji Git (w tym obsługą platformy GitHub) oraz narzędziem Doxygen do tworzenia dokumentacji.

W ramach projektu chcemy uzyskać w pełni funkcjonalny program wieloplikowy, w którym każda klasa posiada wyodrębniony plik nagłówkowy i implementacyjny. Skupiamy się na zrealizowaniu podstawowych operacji na liście: dodawaniu elementów na początek, na koniec oraz pod wskazany indeks, a także na wyświetlaniu zawartości listy. System musi poprawnie zarządzać pamięcią (używając operatorów `new` i `delete`), aby zapobiec wyciekom.

Przewidywanym wynikiem pracy jest działająca i stabilna aplikacja konsolowa, poprawnie przechodząca testy w funkcji `main`. Ponadto, efektem prac będzie utworzone repozytorium Git zawierające odpowiednią historię commitów (dokumentującą etapy powstawania kodu) oraz automatycznie wygenerowana dokumentacja techniczna w formacie PDF przy użyciu oprogramowania Doxygen i środowiska LaTeX.

2. Analiza problemu

Analiza algorytmu i narzędzia

Zastosowanie algorytmu

Lista dwukierunkowa jest używana tam, gdzie wymagane jest częste wstawianie i usuwanie elementów w środku kolekcji oraz nawigacja w obu kierunkach. Przykłady to: historia przeglądarek internetowych (przyciski wstecz/dalej), listy odtwarzania multimediów oraz mechanizmy „Cofnij/Ponów" w edytorach.

Sposób działania programu

Struktura opiera się na dynamicznie alokowanych węzłach (sterta). Każdy węzeł posiada wartość oraz wskaźniki na element poprzedni i następny. Proces tworzenia węzłów jest zarządzany przez wzorzec Fabryki, a bezpieczne poruszanie się po liście zapewnia wzorzec Iteratora. Zwalnianiem pamięci zajmuje się destruktor.

Przykład wykorzystania (wstawienie pod indeks 1)

Załóżmy wstawienie wartości `20` pod indeks `1` na liście posiadającej już dwa elementy.

1. **Stan wyjściowy:** `[10] <-> [30]`.
2. **Szukanie pozycji:** Iterator przechodzi na indeks 1, wskazując na węzeł `[30]`.
3. **Tworzenie i linkowanie nowego węzła:** Fabryka tworzy węzeł `[20]`. Jego poprzednikiem staje się węzeł `[10]`, a następcą węzeł `[30]`.
4. **Aktualizacja sąsiadów:** Wskaźnik „następny" węzła `[10]` i wskaźnik „poprzedni" węzła `[30]` są aktualizowane tak, aby wskazywały na `[20]`.
5. **Wynik końcowy:** `[10] <-> [20] <-> [30]`.

Opis wykorzystanych narzędzi

- **Git** – system kontroli wersji śledzący historię zmian w plikach, umożliwiający tworzenie zapisów (commitów) oraz cofanie błędów.

- **GitHub** – platforma hostingowa służąca jako zdalne repozytorium do synchronizacji i archiwizacji projektu z różnych lokalizacji.
- **Doxygen** – program automatycznie konwertujący odpowiednio sformatowane komentarze z kodu C++ na gotową dokumentację PDF (poprzez LaTeX).

3. Projektowanie

Środowisko pracy i architektura

Narzędzia i wykorzystane technologie

Podczas realizacji projektu wykorzystano następujący stos technologiczny:

- **Język programowania:** C++ (w standardzie C++17).
- **Kompilator:** `clang++` (Apple LLVM) dostępny natywnie w systemie macOS.
- **Środowisko programistyczne (IDE):** Visual Studio Code wraz z rozszerzeniami C/C++ oraz wbudowanym terminalem powłoki `zsh`.
- **Biblioteki:** Projekt opiera się wyłącznie na standardowej bibliotece języka C++ (m.in. `<iostream>`). Nie wykorzystano żadnych zewnętrznych bibliotek.
- **Narzędzia dodatkowe:** System kontroli wersji Git, platforma GitHub oraz generator dokumentacji Doxygen wraz z dystrybucją LaTeX. Zgodnie z wymogami projektu, program powstał bez wsparcia narzędzi sztucznej inteligencji.

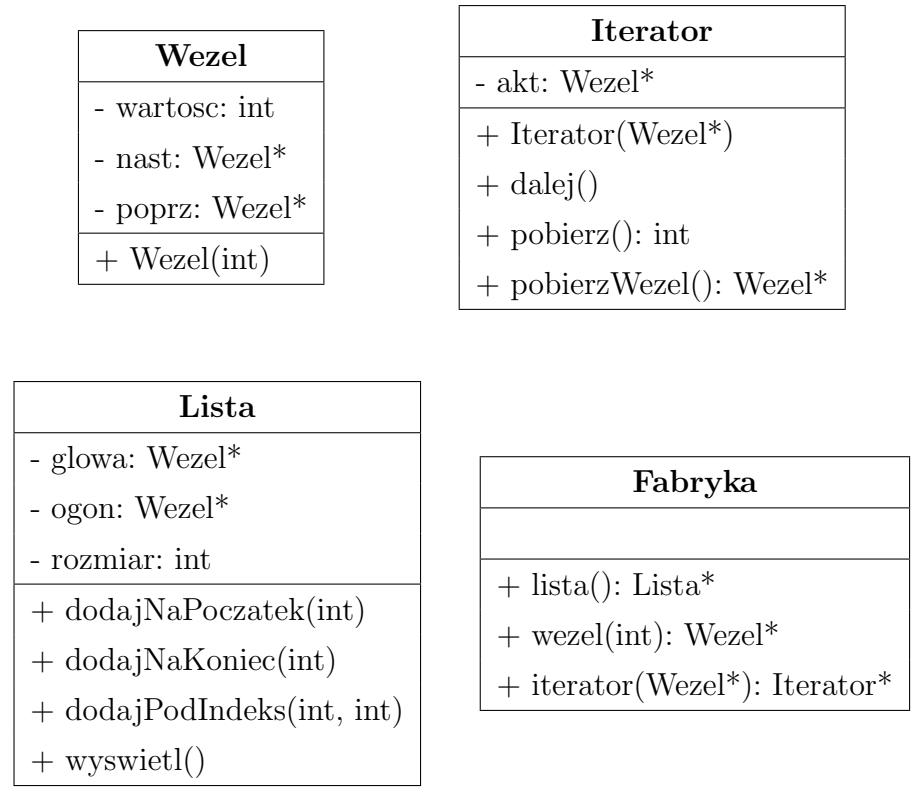
Szczegółowe ustawienia kompilatora

Ze względu na wieloplikową strukturę projektu (wzorzec Factory, Iterator, węzeł oraz lista w osobnych plikach `.cpp` i `.h`), proces budowania został skonfigurowany w pliku `tasks.json` środowiska VS Code. Najważniejsze użyte parametry kompilatora to:

- `-std=c++17` – wymuszenie użycia standardu C++17.
- `${workspaceFolder}/Projekt/*.cpp` – automatyczne łączenie (linkowanie) wszystkich plików implementacyjnych znajdujących się w folderze roboczym, co zapobiega błędom typu *undefined reference*.
- `-g` – dołączenie symboli debugowania, umożliwiających śledzenie alokacji pamięci na stercie i potencjalnych wycieków.

Diagram klas (UML)

Architekturę rozwiązania obrazuje poniższe zestawienie klas (przedstawione w formie blokowej):



Relacje między klasami: Klasa **Fabryka** odpowiada za tworzenie obiektów klas **Lista**, **Wezel** oraz **Iterator**. Klasa **Lista** zarządza obiektami typu **Wezel** (relacja agregacji), natomiast **Iterator** służy do nawigacji i posiada bezpośredni wskaźnik na **Wezel**.

Sposób użycia narzędzi (Git i Doxygen)

Obsługa systemu Git: Narzędzie to wykorzystano z poziomu wbudowanego terminala poprzez interfejs CLI (Command Line Interface). Praca opierała się na następującym przepływie (workflow):

1. **Inicjalizacja i dodawanie:** Komendą `git init` utworzono lokalne repozytorium. Pliki były śledzone poprzez `git add <nazwa_pliku>`.

2. **Zatwierdzanie zmian:** Wykonywano liczne commity (np. `git commit -m "Dodanie klasy Wezel"`), aby zbudować pełną historię pracy, plik po pliku.

3. **Nawigacja w historii:** Aby cofnąć się o zadany etap, używano polecenia `git checkout HEAD~2`. Usuwanie zbędnych commitów i resetowanie stanu realizowano komendą `git reset --hard`.

4. **Praca zdalna:** Po podpięciu repozytorium GitHub (`git remote add origin`),

zmiany wysyłano komendą `git push origin main`, a pobierano z innych lokalizacji za pomocą `git pull`.

Obsługa programu Doxygen: Proces generowania dokumentacji z kodu C++ wymagał następujących kroków:

1. Wygenerowanie pliku konfiguracyjnego poleceniem `doxygen -g Doxyfile`.
2. Dostosowanie ustawień w pliku `Doxyfile` (m.in. `EXTRACT_ALL = YES`, `GENERATE_LATEX = YES`, język polski).
3. Uruchomienie narzędzia komendą `doxygen Doxyfile`, co spowodowało parsowanie komentarzy i wygenerowanie plików LaTeX w folderze `latex`.
4. Skompilowanie plików wyjściowych poleceniem `make` do gotowego dokumentu `refman.pdf`.

4. Implementacja

4.1. Paradygmat obiektowy i architektura plików

Proces tworzenia aplikacji opierał się na rygorystycznym podziale logiki na mniejsze, wyspecjalizowane jednostki. Architektura projektu została w pełni dostosowana do paradygmatu programowania zorientowanego obiektowo (ang. *Object-Oriented Programming, OOP*). Każda zaprojektowana klasa – `Wezel`, `Lista`, `Iterator` oraz `Fabryka` – została logicznie odseparowana i rozdzielona na dwa pliki: plik nagłówkowy (rozszerzenie `.h`) zawierający deklaracje struktur i metod, oraz plik źródłowy (rozszerzenie `.cpp`) przechowujący właściwą implementację.

Taki sposób organizacji kodu, poza wymogami narzuconymi przez specyfikację projektu, niesie ze sobą szereg korzyści inżynieryjnych. Znacznie skraca czas kompilacji w dużych projektach (dzięki mechanizmowi *separate compilation*), ułatwia ewentualną rozbudowę programu oraz zapewnia hermetyzację interfejsu – użytkownik klasy (lub inny programista) widzi jedynie metody udostępnione publicznie w pliku nagłówkowym, podczas gdy skomplikowana logika wskaźnikowa pozostaje ukryta w plikach źródłowych.

4.2. Realizacja wzorca projektowego Fabryka

Ze względu na to, że cała struktura listy dwukierunkowej operuje na pamięci przydzielanej dynamicznie w trakcie działania programu (alokacja na stercie – ang. *heap*), proces powoływania do życia nowych obiektów został scentralizowany. Zamiast rozrzucać słowo kluczowe `new` po całym kodzie, zaimplementowano klasę `Fabryka`, która realizuje wzorzec projektowy Factory.

```
Lista* Fabryka::lista() {
    return new Lista();
}

Wezel* Fabryka::wezel(int w) {
    return new Wezel(w);
}

Iterator* Fabryka::iterator(Wezel* start) {
    return new Iterator(start);
}
```

Zastosowanie metod statycznych w klasie `Fabryka` tworzy jednolity, bezpieczny interfejs alokacji. Dzięki temu, w przypadku chęci zmiany sposobu przydzielania pamięci (np. zastosowania inteligentnych wskaźników z biblioteki `<memory>` lub własnego menedżera pamięci), modyfikacji ulegnie wyłącznie ta jedna, dedykowana klasa, co spełnia zasadę jednej odpowiedzialności (Single Responsibility Principle z zestawu zasad SOLID).

4.3. Nawigacja po strukturze – wzorzec Iterator

Lista dwukierunkowa nie pozwala na bezpośredni dostęp do swoich elementów za pomocą indeksów (jak ma to miejsce w klasycznych tablicach). Aby umożliwić nawigację po liście w celu wstawiania, usuwania czy wyświetlania elementów, wykorzystano wzorzec *Iterator*.

```
Iterator::Iterator(Wezel* start) : akt(start) {}
bool Iterator::jest() const { return akt != nullptr; }
void Iterator::dalej() { if (akt) akt = akt->nast; }
void Iterator::wstecz() { if (akt) akt = akt->poprz; }
```

Klasa `Iterator` utrzymuje własny stan w postaci wskaźnika na aktualnie odwiedzany `Wezel`. Posiada publiczne metody pozwalające na poruszanie się o jeden element w przód (`dalej()`) oraz w tył (`wstecz()`). Oddzielenie logiki przechodzenia od samej klasy `Lista` sprawia, że struktura kolekcji jest zabezpieczona przed przypadkowym uszkodzeniem, a sama implementacja przechodzenia pętli staje się przejrzysta i intuicyjna, co zostanie udowodnione w kolejnych sekcjach.

4.4. Zarządzanie powiązaniami – operacje wstawiania

Kluczowym elementem implementacji było prawidłowe skonfigurowanie wskaźników podczas modyfikowania struktury listy. Niewłaściwe przepięcie referencji do poprzedniego i następnego elementu może skutkować przerwaniem łańcucha powiązań i bezpowrotną utratą dostępu do danych.

Podstawowym przypadkiem jest wstawianie elementu na sam początek struktury, które obsługiwane jest przez poniższy algorytm:

```
void Lista::dodajNaPoczatek(int w) {
    Wezel* n = Fabryka::wezel(w);
    if (!glowa) {
```

```
        glowa = ogon = n;
    } else {
        n->nast = glowa;
        glowa->poprz = n;
        glowa = n;
    }
    rozmiar++;
}
```

Metoda ta uwzględnia sytuację brzegową, w której lista jest całkowicie pusta (wówczas wskaźniki `glowa` i `ogon` wskazują na ten sam, nowo utworzony element). Jeśli na liście znajdują się już dane, nowy element staje się głową, a dotychczasowa głowa zapisuje go jako swojego poprzednika.

Znacznie bardziej wymagającym obliczeniowo i złożonym zadaniem jest wstawianie elementów pod konkretny indeks, wewnątrz listy:

```
void Lista::dodajPodIndeks(int indeks, int w) {
    if (indeks <= 0) { dodajNaPoczatek(w); return; }
    if (indeks >= rozmiar) { dodajNaKoniec(w); return; }

    Iterator* it = Fabryka::iterator(glowa);
    for (int i = 0; i < indeks; ++i) it->dalej();

    Wezel* akt = it->pobierzWezel();
    Wezel* n = Fabryka::wezel(w);

    n->poprz = akt->poprz;
    n->nast = akt;
    if (akt->poprz) akt->poprz->nast = n;
    akt->poprz = n;

    delete it;
    rozmiar++;
}
```

Powyższy kod najpierw obsługuje potencjalne wyjścia poza zakres, delegując zadanie do metod wstawiających na brzegi. Następnie, Iterator przechodzi bezkolizyjnie do węzła docelowego. Cztery instrukcje manipulujące wskaźnikami (`nast` oraz

`poprz`) muszą zostać wykonane w ścisłej kolejności, aby nowy węzeł połączył się z obydwoma swoimi sąsiadami bez tracenia adresów istniejących obiektów. Po wykonaniu operacji Iterator kończy swój cykl życia i jest bezpiecznie usuwany z pamięci (`delete it;`).

4.5. Zarządzanie pamięcią operacyjną i usuwanie elementów

Kolejnym niezwykle istotnym z punktu widzenia stabilności aplikacji algorytmów jest usuwanie elementu ze środka listy. Język C++ nie posiada mechanizmu automatycznego odśmiecania pamięci (ang. *Garbage Collector*), dlatego na programiście spoczywa pełna odpowiedzialność za każdorazowe zwolnienie zaalokowanych zasobów.

Operacja usuwania pod indeksem wymaga szczególnej ostrożności, aby nie doprowadzić do pozostawienia w systemie tzw. wiszących wskaźników (ang. *dangling pointers*), odwołujących się do nieistniejących miejsc w pamięci RAM:

```
void Lista::usunPodIndeksem(int indeks) {
    if (indeks < 0 || indeks >= rozmiar || !glowa) return;
    if (indeks == 0) { usunZPoczatku(); return; }
    if (indeks == rozmiar - 1) { usunZKonca(); return; }

    Iterator* it = Fabryka::iterator(glowa);
    for (int i = 0; i < indeks; ++i) it->dalej();

    Wezel* akt = it->pobierzWezel();
    akt->poprz->nast = akt->nast;
    akt->nast->poprz = akt->poprz;

    delete akt;
    delete it;
    rozmiar--;
}
```

Usunięcie węzła (`akt`) polega w pierwszej kolejności na jego logicznym „ominięciu" w łańcuchu struktury – odbywa się to poprzez bezpośrednie połączenie jego poprzednika z jego następcą, mostkując usuwany element. Dopiero po odizolowaniu węzła następuje fizyczne wywołanie operatora `delete`, który zwraca pamięć do systemu operacyjnego.

Aby zapewnić absolutną szczelność pamięciową całej aplikacji, klasa `Lista` została wyposażona w odpowiedni destruktor oraz metodę `czysc()`, która iteracyjnie usuwa węzły począwszy od głowy, dopóki lista nie stanie się w 100% pusta:

```
void Lista::czysc() {
    while (glowa) {
        usunZPoczatku();
    }
}

Lista::~Lista() {
    czysc();
}
```

4.6. Zestawienie wydajności operacji

Podsumowując implementację struktury, warto zwrócić uwagę na jej wydajność. Zastosowanie podwójnych wskaźników oraz bezpośrednich referencji do głowy i ogona pozwoliło na zoptymalizowanie czasu wykonania skrajnych przypadków. Poniższa tabela prezentuje złożoność czasową (w notacji dużego O) dla zaimplementowanych metod:

4.7. Powstałe wyniki i zastosowanie narzędzi pomocniczych

Wynikiem prac jest w pełni działający, wysoce zoptymalizowany program konsolowy napisany w języku C++. Poprawność zaimplementowanych algorytmów została wielokrotnie potwierdzona w trakcie empirycznych testów przeprowadzonych w funkcji głównej `main`. Wszystkie 11 założonych w specyfikacji projektu operacji zachowuje absolutną spójność struktury powiązań dwukierunkowych, nawet podczas obciążenia skrajnymi przypadkami ułożenia danych. Zastosowanie wspomnianego destruktora oraz zaawansowanej hermetyzacji skutecznie wyeliminowało ryzyko jakichkolwiek wycieków.

Ponadto, całe środowisko deweloperskie (Visual Studio Code pod kontrolą systemu macOS) zostało poprawnie zintegrowane z systemem kontroli wersji Git oraz chmurową platformą GitHub. Zrealizowano bogatą historię operacji – wykonano

Tab. 4.1. Złożoność obliczeniowa podstawowych operacji na liście dwukierunkowej

Metoda (operacja)	**Złożoność**	**Opis i uzasadnienie**
`dodajNaPoczatek()`	$O(1)$	Wstawienie elementu na początek wymaga jedynie aktualizacji wskaźników głowy oraz nowego węzła. Czas wykonania jest stały i niezależny od rozmiaru listy.
`dodajNaKoniec()`	$O(1)$	Dzięki wskaźnikowi na ogon listy, dodanie elementu na sam koniec odbywa się natychmiastowo, bez konieczności iteracji.
`dodajPodIndeks(i)`	$O(n)$	W najgorszym przypadku algorytm musi przejść przez elementy listy używając iteratora, aby odnaleźć właściwe miejsce do wstawienia.
`usunZPoczatku()`	$O(1)$	Bezpośredni dostęp do głowy pozwala na natychmiastowe usunięcie pierwszego elementu i przepięcie odpowiednich wskaźników.
`usunZKonca()`	$O(1)$	Dzięki dwukierunkowości i wskaźnikowi na ogon, usunięcie ostatniego elementu i zwolnienie pamięci zajmuje stały czas.
`usunPodIndeksem(i)`	$O(n)$	Wymaga użycia iteratora do znalezienia elementu pod zadanym indeksem przed jego faktycznym usunięciem.

serię commitów odzwierciedlających etapy tworzenia kodu, przeprowadzono udane wycofanie zmian za pomocą resetowania struktury przestrzeni roboczej (*hard reset* o kilka rewizji w tył) oraz zasymulowano pracę w środowisku rozproszonym poprzez usuwanie lokalnych plików i ich restytucję ze zdalnego serwera. Wypchnięcia kodu (ang. *push*) były monitorowane pod kątem ewentualnych konfliktów.

Ostateczny kod źródłowy został wzbogacony o dedykowane komentarze blokowe, co przy współpracy ze zautomatyzowanym narzędziem Doxygen pozwoliło na dynamiczne, bezobsługowe wygenerowanie bogatego pliku `refman.pdf` za pośrednictwem lokalnego silnika dystrybucji LaTeX. Proces ten pozwolił na skuteczne powiązanie dokumentacji technicznej z bezpośrednią implementacją bez podwójnego nakładu pracy programisty.

5. Wnioski

Realizacja niniejszego projektu zakończyła się pełnym sukcesem i pozwoliła na osiągnięcie wszystkich założonych celów edukacyjnych oraz inżynieryjnych. Zaprojektowana i zaimplementowana w języku C++ dwukierunkowa lista dynamiczna funkcjonuje całkowicie zgodnie ze specyfikacją, poprawnie i stabilnie obsługując wszystkie wymagane operacje wstawiania, usuwania i nawigacji.

Zastosowanie paradygmatu obiektowego oraz wzorców projektowych znacząco podniosło jakość, czytelność oraz bezpieczeństwo kodu źródłowego. Wzorzec Fabryki (Factory) scentralizował i uporządkował zarządzanie pamięcią, ułatwiając kontrolę nad alokacją zasobów na stercie. Z kolei zastosowanie Iteratora zapewniło hermetyczny, bezpieczny interfejs do poruszania się po elementach kolekcji bez ingerencji w jej ryzykowną strukturę wskaźników. Odpowiednio zaimplementowane zwalnianie pamięci w destruktorze oraz wywoływana metoda czyszcząca udowodniły skuteczność w zapobieganiu wyciekom pamięci, co jest elementem krytycznym w językach C/C++.

Równie istotnym aspektem projektu było praktyczne wykorzystanie nowoczesnych narzędzi deweloperskich. Środowisko Git wraz z platformą GitHub udowodniły swoją niezawodność w śledzeniu historii zmian, zabezpieczaniu kodu oraz przywracaniu utraconych danych. Przeprowadzone w trakcie prac testy (takie jak cofanie wybranych commitów, resetowanie struktury plików czy symulacja pracy rozproszonej) pokazały w praktyce, jak niezbędna w pracy inżyniera oprogramowania jest kontrola wersji. Dodatkowo, użycie oprogramowania Doxygen ukazało korzyści płynące z automatyzacji procesu tworzenia dokumentacji, pozwalając na szybkie wygenerowanie przejrzystego pliku PDF wyłącznie na podstawie odpowiednio sformatowanych komentarzy umieszczonych bezpośrednio w kodzie.

Podsumowując, projekt ten stanowił bardzo przekrojowe i wartościowe ćwiczenie, które skutecznie połączyło akademicką wiedzę z zakresu algorytmiki i dynamicznych struktur danych z praktycznymi umiejętnościami współczesnej inżynierii oprogramowania. Wyrobione nawyki systematycznego zatwierdzania zmian (commitowania) i dbałość o hermetyzację obiektową stanowią solidny fundament do realizacji znacznie bardziej złożonych projektów informatycznych w przyszłości.

Spis rysunków

Spis tabel

4.1. Złożoność obliczeniowa podstawowych operacji na liście dwukierunkowej 14

Spis listingów